C1W2_UGL_solving_linear_systems_3_variables

December 2, 2025

1 Introduction to the Numpy.linalg sub-library

In this lab, you will keep developing your skills using Python to solve systems of linear equations by using the sub-library numpy.linalg. In this notebook you will:

- Use `NumPy` linear algebra package to find the solutions to the system of linear equations
- Evaluate the determinant of the matrix to see again the connection between matrix singularity and the number of solutions of the linear system
- Explore the numpy.linalg sub-library to get to know its properties

2 Table of Contents

2.1 Packages

Load the `NumPy` package to access its functions.

[2]:
```
import numpy as np
```

1 - Representing and Solving a System of Linear Equations using Matrices

1.1 - System of Linear Equations

Here is a **system of linear equations** (or **linear system**) with three equations and three unknown variables:

$$\begin{cases} 4x_1 - 3x_2 + x_3 = -10, \\ 2x_1 + x_2 + 3x_3 = 0, \\ -x_1 + 2x_2 - 5x_3 = 17, \end{cases} \tag{1}$$

To solve this system of linear equations means to find such values of the variables x_1, x_2, x_3, that all of its equations are simultaneously satisfied.

1.2 - Solving Systems of Linear Equations using Matrices

Let's prepare to solve the linear system (1) using `NumPy`. A will be a matrix, each row will represent one equation in the system and each column will correspond to the variable x_1, x_2, x_3. And b is a 1-D array of the free (right side) coefficients:

[3]:
```
A = np.array([
        [4, -3, 1],
        [2, 1, 3],
        [-1, 2, -5]
    ], dtype=np.dtype(float))

b = np.array([-10, 0, 17], dtype=np.dtype(float))

print("Matrix A:")
print(A)
print("\nArray b:")
print(b)
```

```
Matrix A:
[[ 4. -3.  1.]
 [ 2.  1.  3.]
 [-1.  2. -5.]]

Array b:
[-10.   0.  17.]
```

Check the dimensions of A and b using `shape()` function:

[4]:
```
print(f"Shape of A: {np.shape(A)}")
print(f"Shape of b: {np.shape(b)}")
```

```
Shape of A: (3, 3)
Shape of b: (3,)
```

Now use `np.linalg.solve(A, b)` function to find the solution of the system (1). The result will be saved in the 1-D array x. The elements will correspond to the values of x_1, x_2 and x_3:

[5]:
```
x = np.linalg.solve(A, b)

print(f"Solution: {x}")
```

```
Solution: [ 1.  4. -2.]
```

Try to substitute those values of x_1, x_2 and x_3 into the original system of equations to check its consistency.

1.3 - Evaluating the Determinant of a Matrix

Matrix A corresponding to the linear system (1) is a **square matrix** - it has the same number of rows and columns. In the case of a square matrix it is possible to calculate its determinant - a real number that characterizes some properties of the matrix. A linear system containing three

equations with three unknown variables will have one solution if and only if the matrix A has a non-zero determinant.

Let's calculate the determinant using `np.linalg.det(A)` function:

[6]:
```
d = np.linalg.det(A)

print(f"Determinant of matrix A: {d:.2f}")
```

```
Determinant of matrix A: -60.00
```

Please note that its value is non-zero, as expected.

1.4 - What happens if the system has no unique solution?

Let's explore what happens if we use `np.linalg.solve` in a system with no unique solution (no solution at all or infinitely many solutions).

Given another system of linear equations:

$$\begin{cases} x_1 + x_2 + x_3 = 2, \\ x_2 - 3x_3 = 1, \\ 2x_1 + x_2 + 5x_3 = 0, \end{cases} \tag{2}$$

let's find the determinant of the corresponding matrix.

[7]:
```
A_2= np.array([
        [1, 1, 1],
        [0, 1, -3],
        [2, 1, 5]
    ], dtype=np.dtype(float))

b_2 = np.array([2, 1, 0], dtype=np.dtype(float))

print(np.linalg.solve(A_2, b_2))
```

```
---------------------------------------------------------------------------
LinAlgError                               Traceback (most recent call last)
<ipython-input-7-a5db2e8e2011> in <module>
      7 b_2 = np.array([2, 1, 0], dtype=np.dtype(float))
      8
----> 9 print(np.linalg.solve(A_2, b_2))

<__array_function__ internals> in solve(*args, **kwargs)

/opt/conda/lib/python3.8/site-packages/numpy/linalg/linalg.py in solve(a, b)
    391     signature = 'DD->D' if isComplexType(t) else 'dd->d'
    392     extobj = get_linalg_error_extobj(_raise_linalgerror_singular)
--> 393     r = gufunc(a, b, signature=signature, extobj=extobj)
    394
```

```
    395     return wrap(r.astype(result_t, copy=False))

/opt/conda/lib/python3.8/site-packages/numpy/linalg/linalg.py in␣
 ↪_raise_linalgerror_singular(err, flag)
     86
     87 def _raise_linalgerror_singular(err, flag):
---> 88     raise LinAlgError("Singular matrix")
     89
     90 def _raise_linalgerror_nonposdef(err, flag):

LinAlgError: Singular matrix
```

As you can see, it throws a `LinAlgError` because the matrix is singular. You can check this for yourself by computing the determinant of the matrix:

[8]:
```
d_2 = np.linalg.det(A_2)

print(f"Determinant of matrix A_2: {d_2:.2f}")
```

```
Determinant of matrix A_2: 0.00
```

The sub-library np.linalg has several linear algebra functions to help linear algebra tasks and we have exhausted so far the functions you have learned in class.

Once you learn more theory, the functions in this library will become clearer. You also will be using them in the assignments in week 3 and 4, but do not worry because you will be guided through them.

Well done! You used the `NumPy` functions to solve a system of equations. As expected, using a predefined function is much easier, but gives much less insight into what is happening under the hood. **This is why the next assignment will be in Gaussian Elimination, a method to solve linear systems**. Remember that np.linalg.solve gives an error if there are no or infinitely many solutions. When using it, you will need to check for that case to prevent your program from crashing.

[ ]:
```

```